



PES

UNIVERSITY

CELEBRATING 50 YEARS

PROGRAMMING WITH PYTHON

Lekha A

Computer Applications

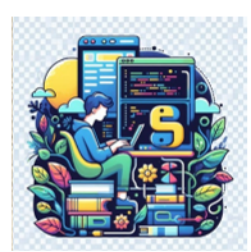
PROGRAMMING WITH PYTHON

Object-Oriented Programming (OOP) and Advanced Concepts

Metaclasses

Lekha A

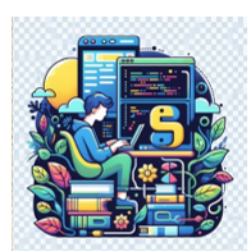
Computer Applications



PROGRAMMING WITH PYTHON

Metaprogramming

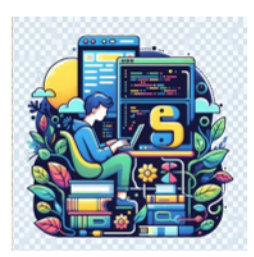
- Metaprogramming refers to the writing of code that manipulates, creates, or deals with other pieces of code, such as functions, classes, or modules, during runtime.
- In Python, metaprogramming involves using the language's introspection capabilities, dynamic nature, and features like decorators, metaclasses, and reflection to modify or generate code at runtime.



PROGRAMMING WITH PYTHON

Key aspects in Metaprogramming

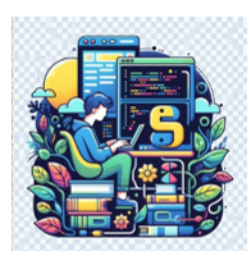
- Decorators
 - Functions that wrap other functions or classes, modifying their behavior without changing their code directly.
 - Used for tasks like logging, timing, caching, access control, error handling, validation, and more.



PROGRAMMING WITH PYTHON

Key aspects in Metaprogramming

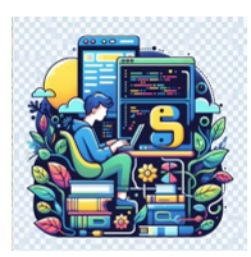
- Metaclasses
 - Classes that create classes.
 - They control the creation and behavior of classes themselves.
 - Act as blueprints for classes, defining how classes are instantiated and initialized.
 - Allow customization of class creation and behavior.



PROGRAMMING WITH PYTHON

Key aspects in Metaprogramming

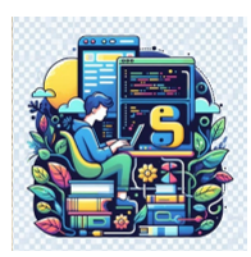
- Decorators
 - Functions that wrap other functions or classes, modifying their behavior without changing their code directly.
 - Used for tasks like logging, timing, caching, access control, error handling, validation, and more.



PROGRAMMING WITH PYTHON

Key aspects in Metaprogramming

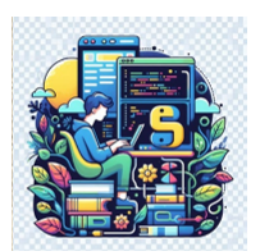
- Decorators
 - Functions that wrap other functions or classes, modifying their behavior without changing their code directly.
 - Used for tasks like logging, timing, caching, access control, error handling, validation, and more.



PROGRAMMING WITH PYTHON

Key aspects in Metaprogramming

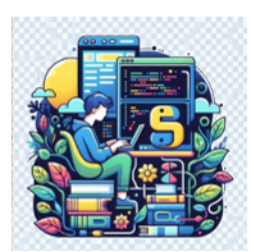
- Introspection
 - Examining the structure and characteristics of live objects at runtime.
 - Involves using introspection functions like `dir()`, `type()`, `getattr()`, `hasattr()`, and `isinstance()`.



PROGRAMMING WITH PYTHON

Key aspects in Metaprogramming

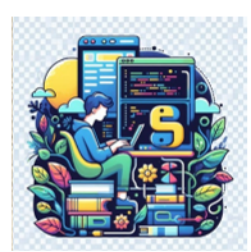
- Dynamic Code Execution
 - Executing code generated as strings or from external sources.
 - Uses functions like `exec()`, `eval()`, and `compile()`.
- Dynamic Attributes and Functions
 - Python's dynamic nature allows for the addition or modification of attributes and methods to classes or instances during runtime.



PROGRAMMING WITH PYTHON

Key aspects in Metaprogramming

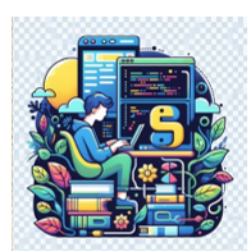
- Dynamic Code Execution
 - Executing code generated as strings or from external sources.
 - Uses functions like `exec()`, `eval()`, and `compile()`.
- Dynamic Attributes and Functions
 - Python's dynamic nature allows for the addition or modification of attributes and methods to classes or instances during runtime.



PROGRAMMING WITH PYTHON

Metaclass

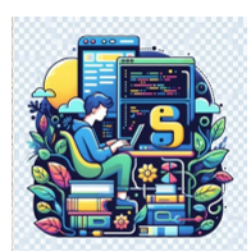
- A Class is also an object, and just like any other object, it's an instance of something called Metaclass.
- A special class type creates these Class objects.
- The type class is default metaclass which is responsible for making classes.



PROGRAMMING WITH PYTHON

Metaclass

- A metaclass in Python is a class of a class that defines how a class behaves.
- A class is itself an instance of a metaclass.
- A class in Python defines how the instance of the class will behave.



PROGRAMMING WITH PYTHON

Metaclass

```
class AutoPrinter(type):  
    def __new__(cls, name, bases, dct):  
        print(f"Creating class: {name}")  
        return super().__new__(cls, name, bases, dct)
```

Using the metaclass in a class definition

```
class Student(metaclass=AutoPrinter):  
    def __init__(self, name):  
        self.name = name  
  
    def greet(self):  
        print(f"Hello, I'm {self.name}")
```

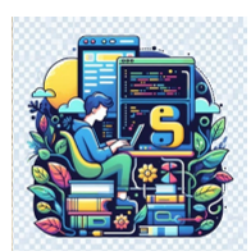
Creating class: Student

```
s = Student("Lee")  
s.greet()
```

Hello, I'm Lee

```
type(s)
```

```
__main__.Student
```



PROGRAMMING WITH PYTHON

Metaclass

- Using Type

```
def greet(self):  
    print(f"Hello, {self.name}")
```

```
Student = type("Student", (object,), {"greet": greet, "name": "Lily"})
```

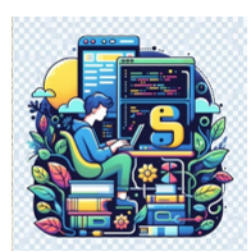
```
s = Student()
```

```
type(s)
```

```
__main__.Student
```

```
s.greet()
```

```
Hello, Lily
```



PROGRAMMING WITH PYTHON

Metaclass

- In Python, the `__class__` attribute enables the type of the current instance to be verified.

```
concept = 'metaclasses'
```

```
concept.__class__
```

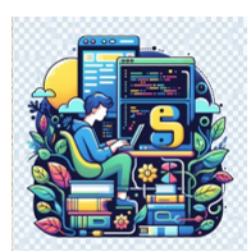
```
str
```

```
type(concept)
```

```
str
```

```
type(str)
```

```
type
```



PROGRAMMING WITH PYTHON

Metaclass

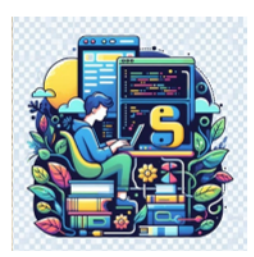
- ```
print(type(list),type(float), type(dict), type(tuple), type(set), type(int))
```

```
<class 'type'> <class 'type'> <class 'type'> <class 'type'> <class 'type'> <class 'type'>
```
- ```
concept.__class__.__class__
```



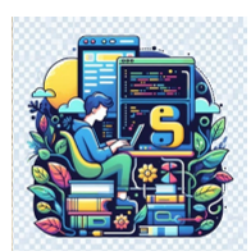
```
type
```

PROGRAMMING WITH PYTHON

Creating Custom Metaclass

- Customize the class creation process by passing the metaclass keyword in the class definition.
- This can also be done by inheriting a class that has already passed in this keyword.

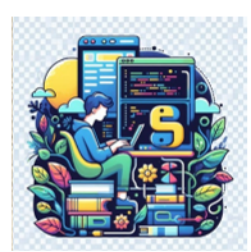


PROGRAMMING WITH PYTHON

Creating Custom Metaclass

- Metaclass

```
class CustomMeta(type):  
    def __new__(cls, name, bases, dct):  
        print(f"Creating class: {name}")  
        # Add a new attribute dynamically  
        dct['created_by'] = "CustomMeta"  
        return super().__new__(cls, name, bases, dct)
```



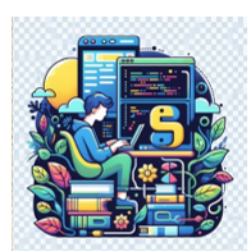
PROGRAMMING WITH PYTHON

Creating Custom Metaclass

Using metaclass keyword directly

```
class ExampleA(metaclass=CustomMeta):  
    def greet(self):  
        print("Hello from ExampleA!")
```

Creating class: ExampleA



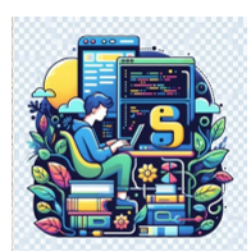
PROGRAMMING WITH PYTHON

Creating Custom Metaclass

Inheriting from a class that uses the metaclass

```
class ExampleB(ExampleA):  
    def greet(self):  
        print("Hello from ExampleB!")
```

Creating class: ExampleB



PROGRAMMING WITH PYTHON

Creating Custom Metaclass

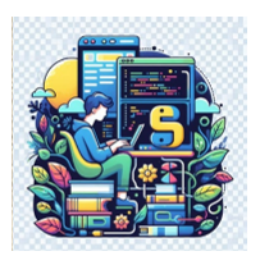
```
a = ExampleA()  
b = ExampleB()  
a.greet()  
b.greet()
```

```
Hello from ExampleA!  
Hello from ExampleB!
```

Attribute added by metaclass

```
print("ExampleA created by:", a.created_by)  
print("ExampleB created by:", b.created_by)
```

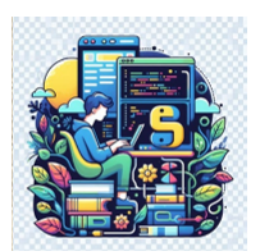
```
ExampleA created by: CustomMeta  
ExampleB created by: CustomMeta
```



PROGRAMMING WITH PYTHON

Creating Custom Metaclass

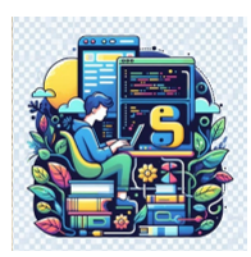
- CustomMeta is a metaclass — it overrides `__new__()` to customize class creation.
- When ExampleA is defined, Python calls CustomMeta.`__new__()`, printing “Creating class: ExampleA”.



PROGRAMMING WITH PYTHON

Creating Custom Metaclass

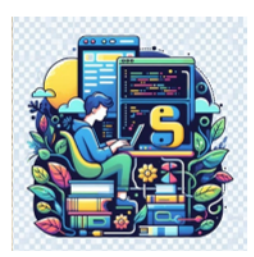
- The metaclass injects a new class attribute `created_by = "CustomMeta"`.
- ExampleB inherits from ExampleA, so it automatically inherits the same metaclass - demonstrating the second method (inheriting from a class that already uses the metaclass keyword).



PROGRAMMING WITH PYTHON

Creating Custom Metaclass

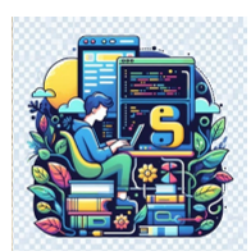
- `__new__` is used when one wants to define dict or bases tuples before the class is created.
- The return value of `__new__` is usually an instance of `cls`.



PROGRAMMING WITH PYTHON

Creating Custom Metaclass

- `__new__` allows subclasses of immutable types to customize instance creation.
- It can be overridden in custom metaclasses to customize class creation.
- `__init__` is usually called after the object has been created so as to initialize it.



PROGRAMMING WITH PYTHON

Recap

- Metaprogramming
- Metaclass
- Custom Metaclass



THANK YOU

Lekha A
Computer Applications